Markdown → PDF Converter (`md2pdf-desktop`)

A Markdown-to-PDF converter designed for technical documents, with mathematical typesetting, Mermaid diagrams, GitHub Flavored Markdown (GFM) alerts, table formatting, and an IDE-inspired Sidebar renderer.

Three operational surfaces:

- **Desktop Studio** (`md2pdf_app.py`) – A modern desktop GUI built with `ui-ux-pro-max` design standards, live toolchain telemetry, real-time syntax auto-detection, and document metrics.
- **MCP Server** (`mcp_server/server.py`) – FastMCP server exposing headless conversion tools to Google Antigravity, Claude Code, and Claude Desktop.
- **PowerShell Sync Engine** (`sync.ps1`) – Automated git synchronization, pre-commit secret scanning, and toolchain health validation.

All surfaces share conversion functions in `md2pdf/core.py` . Explicit renderer modes intentionally produce different styling; automatic backend selection is centralized in the shared core. Phase 3 adds semantic PDF parsing and a maintained release regression corpus so artifact correctness is tested in addition to process success.

CONVERSION ENGINES

1. Sidebar Mode (Default & Recommended)

Pipeline: `pandoc` (MD → HTML5 AST) → locally provisioned KaTeX + Mermaid assets → Headless Chromium (Google Chrome / Microsoft Edge) → Vector PDF.

- **Visual Match**: Renders Markdown with visual fidelity matching the modern IDE Markdown preview sidebar (Google Antigravity, VS Code, GitHub).
- **Mermaid Vector Diagrams**: Renders ```` ```mermaid ```` flowcharts, sequence diagrams, and state charts directly into vector SVGs.
- **Crisp KaTeX Math**: Renders both inline `$ ... $` and display `$$ ... $$` formulas (matrices, fractions, Greek letters, integrals) without blurry bitmap rasterization.
- **GFM Alert Callouts**: Transforms `> [!TIP]` , `> [!NOTE]` , `> [!WARNING]` , `> [!IMPORTANT]` , and `> [!CAUTION]` into styled boxes with themed accent borders, background tints, and icons.
- **Dual Themes**:
  - **Light Mode (`theme="light"` , Default)**: Pure white canvas (`#ffffff`), dark slate typography (`#111827`), blue question accents (`#1d4ed8`), violet algorithm headers (`#7c3aed`), and light-themed diagram nodes. Ideal for physical paper printing.
  - **Dark Mode (`theme="dark"`)**: Deep zinc palette (`#18181b`) matching dark IDE preview panels.
- **Print Pagination**: Uses Chromium's print layout and page CSS. Complex equations, tables, code, and diagrams should be checked in the generated PDF because browser pagination can still split content.
- **Deterministic Browser Assets**: KaTeX and Mermaid are pinned to exact package versions, downloaded from npm package tarballs with SHA-512 verification, then cached locally. Subsequent offline conversions do not contact a CDN.

2. LaTeX Formal Mode

Pipeline: `pandoc` (MD → LaTeX, via `md2pdf/templates/styled.latex`) → `pdflatex` .

- Native vector LaTeX typesetting with `mathpazo` Palatino typography.
- Monochrome `tcolorbox` -styled callouts (`::: {.callout}` , `::: {.answer}`) and `booktabs` tables.
- Hard line wrapping via `fvextra` (`fontsize=\footnotesize, breaklines=true`) preventing page overflows.
- Default 12.7mm (0.5 in) margins.

3. □ Simple Mode

Pipeline: `pandoc` (MD → HTML) → `wkhtmltopdf` .

- Fast, lightweight HTML/CSS print engine with `--webtex` formula rendering.

4. Auto Mode

Content-aware backend selector. Inspects Markdown content: - If Mermaid diagrams or GFM alerts are detected → requires **Sidebar** mode; Auto reports the specialized renderer failure rather than silently degrading the document to another backend. - If complex LaTeX math or callout divs are detected → routes to **Sidebar** (if Chromium present) or **LaTeX** (if `pdflatex` present). - Otherwise falls back to **Simple** mode. - Renderer output is published atomically, so a failed conversion does not replace an existing valid PDF.

□ COMMAND-LINE INTERFACE

Install the Python package from a checkout (Python 3.10+):

```
python -m pip install .
```

Then convert a document:

```
md2pdf notes.md --mode auto --output notes.pdf
md2pdf notes.md --mode sidebar --theme dark --margin 14mm
```

The CLI reports the selected renderer and returns a non-zero exit code for invalid input or conversion failures. `md2pdf --version` reports the installed package version. Pandoc and the chosen renderer executable remain system dependencies; installing the Python package does not install those tools. MCP server dependencies remain in `requirements.txt`.

Provision or verify the Sidebar browser assets explicitly:

```
python -m md2pdf.web_assets
python -m md2pdf.web_assets --offline
```

Set `MD2PDF_OFFLINE=1` to make Sidebar conversion cache-only. The cache is stored outside the Python installation and is re-hash-verified from its manifest.

DESKTOP STUDIO GUI (`md2pdf_app.py`)

Run the desktop application from source:

```
python md2pdf_app.py
```

Windows release bundle

Tagged releases publish a self-contained Windows x64 `md2pdf Studio` application bundle built with PyInstaller. The executable embeds the Windows Per-Monitor V2 DPI declaration so the packaged process has a release-grade DPI default instead of inheriting `python.exe`'s process DPI state. Native conversion tools such as Pandoc, Chrome/Edge, LaTeX, and wkhtmltopdf are still discovered from the host system.

Studio Highlights

- **`ui-ux-pro-max` Dark Palette**: Deep canvas (`#080c16`), elevated card containers (`#0f172a`), subtle slate borders (`#1e293b`), and vibrant accents.
- **Toolchain Telemetry Badges**: Header pills showing live availability for `Chromium`, `Pandoc`, `LaTeX`, and `wkhtmltopdf`.
- **Dynamic Syntax Auto-Detector**: Automatically analyzes editor text and informs you why a specific engine was chosen.
- **Live Document Stats**: Tracks line count, word count, character count, and file size in real time.
- **Quick Action Bar**: `[Folder] Open .md File`, `Paste Clipboard`, `Clear`, and active file badge.
- **Responsive Conversion**: PDF rendering runs in a background worker so the Tkinter editor remains interactive while conversion is in progress.
- **Export Presets & Automation**: Quick margin dropdown (`10mm`, `14mm`, `20mm`, `0.5in`), auto-open PDF on completion, and `[Folder] Show in Folder` shortcut.

□ *Windows Explorer Context Menu Integration*

Convert Markdown files directly from Windows File Explorer without opening the editor:

```
# Register context menu (Current User, no admin rights required)
.\setup_context_menu.bat

# Unregister / remove anytime
.\setup_context_menu.bat -Unregister
```

Context Menu Feature: - **`Convert to PDF (md2pdf)`** – 1-click direct conversion using Smart Detect; generates `<filename>.pdf` alongside the source document and displays a non-blocking toast notice with `[Open PDF]` and `[Show in Folder]` buttons.

MODEL CONTEXT PROTOCOL (MCP) SERVER

The MCP server allows AI coding agents in Google Antigravity or Claude Desktop to convert Markdown files to PDF autonomously.

Start Standalone:

```
python mcp_server/server.py
```

Tools Exposed:

1. **`convert_markdown_to_pdf`**
   - `markdown` (string): Source Markdown text.
   - `output_path` (string): Target PDF file path.
   - `mode` (`"sidebar"` | `"latex"` | `"simple"` | `"auto"`, default: `"auto"`).
   - `margin_mm` (int, default: 12).
2. **`detect_latex_needed`**

- Inspects Markdown and recommends the optimal backend.

3. `check_conversion_dependencies`
   - Health check returning tool status and availability.

Configuration in Google Antigravity / Claude Desktop:

```
{
  "mcpServers": {
    "md2pdf": {
      "command": "python",
      "args": ["F:/Aaradhya-Dev-Tamrakar/md2pdf-desktop/mcp_server/server.py"]
    }
  }
}
```

System Requirements

- **Python 3.10+**
- **Pandoc** (required for all modes):

  ```
  choco install pandoc
  ```

- **Chromium** (for Sidebar mode – either Google Chrome or Microsoft Edge):
  - Pre-installed on Windows (`msedge.exe` or `chrome.exe`).
  - The first Sidebar conversion provisions pinned browser assets from npm; later runs reuse the local cache.
- **Optional Tools**:
  - `pdflatex` (TeX Live or MiKTeX) for LaTeX Formal mode.
  - `wkhtmltopdf` for Simple mode.

Automated Git Synchronization (`sync.ps1`)

The repository includes a dedicated sync engine enforcing pre-commit secret scans and conventional commit standards:

```
# Routine sync: pull, secret scan, commit, and push
.\sync.ps1

# Sync with custom commit message
.\sync.ps1 -m "feat(sidebar): add light paper print theme"

# Check conversion toolchains & probe template
.\sync.ps1 -CheckTools

# Safe pull only
.\sync.ps1 -PullOnly

# Dry run preview (read-only; does not stage or reset files)
.\sync.ps1 -WhatIf

# Explicitly repair a missing/mismatched origin remote
.\sync.ps1 -RepairRemote
```

[Folder] Repository Structure

- `md2pdf/core.py` – Core conversion pipelines (`convert_sidebar` , `convert_latex` , `convert_simple` , `convert_auto`), AST sanitizers, and CSS templates.
- `md2pdf_app.py` – Modern Tkinter desktop application (`md2pdf Studio`).
- `mcp_server/server.py` – FastMCP server for AI agent workflows.
- `md2pdf/templates/styled.latex` – Pandoc LaTeX template for formal print outputs.
- `md2pdf/templates/callout-boxes.lua` – Lua filter for LaTeX tcolorbox mapping.
- `sync.ps1` – Automated sync engine and secret guard.

Release-Quality Validation

The current release target is version `0.4.0` . Release validation covers the Python 3.10-3.12 matrix, wheel + source-distribution contents, installed CLI smoke tests,

real `Pandoc → wkhtmltopdf` rendering, real offline Sidebar/Chromium rendering, semantic PDF parsing through `pypdf`, and a maintained rendering corpus. See `docs/RELEASE_CHECKLIST.md`.

Release artifacts are built by `release.yml` when a `v*` tag is pushed. Releases include the Python source distribution/wheel and a Windows x64 `md2pdf Studio` desktop bundle. The Windows executable embeds a Per-Monitor V2 application manifest; the project still does not bundle Pandoc, Chromium, TeX, or wkhtmltopdf, which remain documented system dependencies.

RELIABILITY CHECKS

Run the standard-library regression suite:

```
python -m unittest discover -s tests -v
```

The tests cover PDF parsing and artifact integrity, automatic-renderer fallback behavior, Markdown normalization, package installation, GUI worker behavior, and the release regression corpus. Separate integration jobs invoke the real Pandoc + wkhtmltopdf and Chromium Sidebar pipelines.

Reproducibility note

Sidebar mode no longer loads KaTeX or Mermaid from a CDN at render time. The exact KaTeX 0.16.11 and Mermaid 10.9.3 npm package tarballs are SHA-512 verified before the required browser assets are extracted into a local cache. Set `MD2PDF_OFFLINE=1` to forbid network provisioning and require an existing verified cache. Browser, operating-system, and font rendering differences can still affect pixel-level output; the project therefore validates artifact integrity and exercises the real Chromium path in CI rather than claiming byte-for-byte PDF identity.

PHASE 3 SCOPE

Phase 3 is deliberately a release-hardening layer rather than a renderer feature expansion. The goal is to make the current behavior measurable, parse-validated, regression-tested, packaged, and releasable without claiming pixel-identical PDFs across operating systems.